



Lab session 8

Objectives

- Build an application based on the patterns: hexagonal, domain model, and repository with JPA mapper
- Evolve the layering architecture of the previous release version with JDBC to integrate the domain model and mapper

Exercises

Evolve the architecture from the previous version with JDBC

Follow this order:

- Upload to the Eclipse the project that accompanies this practice.
- Copy your domain model classes developed in previous practice to the `uo.ri.cws.domain` package.
- Startup the attached HSQLDB database with the CarWorkshop app data.
- Look at the distribution of projects. There is one with the interfaces of the service layer, several presentation applications (one for each actor in the system; they are already complete) and the project for the services.
- Notice the interfaces of the service layer, they are the same as those of the previous practice. Observe the use of the service, façade, factory and DTO patterns.
- Redo the classes that implement the *Transaction Scripts* (those in the `uo.ri.cws.application.service.*.*` package). They now must be developed using the model classes and repositories.

We will start with the CRUD cases of mechanics and the one for creating invoices.

Centralize transaction control

- Add the `Command` and `CommandExecutor` interfaces.
- All the transaction scripts must implement the `Command` interface.
- Modify the classes that implement the facades so that each TS is executed by the `CommandExecutor` (one line of code for each method).
- Adds the utility class `Jpa.java` to the persistence layer. Look at their content and make sure you know what each of the public methods is for.
- Implement the `CommandExecutor`. Make use of the newly added `Jpa` utility class.
- Adjust the `Factories` class in the `uo.ri.conf` package to get an instance of the `CommandExecutor`.



- Modify each TS by removing redundant code. Remember that TSs are implemented as commands. And these no longer contain the domain logic (it is in the classes of the domain model); Its mission is to validate input parameters, invoke repositories, and delegate to domain classes to execute logic.

Complete repositories

- Look at the package `uo.ri.cws.application.repository`, it contains all the interfaces of the repositories. One for each entity. You'll see that they all inherit from the base interface *Repository*. Repositories use the metaphor of the collection (add and remove, but not update), they are not TDG or DAO.
- The package `uo.ri.cws.infrastructure.persistence.jpa` contains the implementations of those interfaces. There is a base implementation for common methods. You just must fill in the specific queries for each repository. Remember to retrieve the mapper with the utility class *Jpa*.
- Add the necessary queries in repository deployments.
 - * Each method performs a query expressed in JPQL that must be located in the `orm.xml` file.
 - * At least add the *MechanicJpaRepository*, *InvoiceJpaRepository*, and *WorkOrderJpaRepository* with the necessary methods.